boost::asio协程实现并发服务器

Posted on 2023-06-16 14:20:28 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2RHA2vfIlSmYXf4xcJqzzVtLrJt)

简介

之前介绍了asio服务器并发编程的几种模型，包括单线程，多线程IOServicePool，多线程IOThreadPool等，今天带着大家利用asio协程实现并发服务器。利用协程实现并发程序有两个好处

1　将回调函数改写为顺序调用，提高开发效率。

2　协程调度比线程调度更轻量化，因为协程是运行在用户空间的，线程切换需要在用户空间和内核空间切换。

协程案例

asio官网提供了一个协程并发编程的案例，我们列举一下

```
1.
2. #include <boost/asio/co_spawn.hpp>
3. #include <boost/asio/detached.hpp>
4. #include <boost/asio/io_context.hpp>
5. #include <boost/asio/ip/tcp.hpp>
6. #include <boost/asio/signal_set.hpp>
7. #include <boost/asio/write.hpp>
8. #include <cstdio>
9.
10. using boost::asio::ip::tcp;
11. using boost::asio::awaitable;
12. using boost::asio::co_spawn;
13. using boost::asio::detached;
14. using boost::asio::use_awaitable;
15. namespace this_coro = boost::asio::this_coro;
16.
17. #if defined(BOOST_ASIO_ENABLE_HANDLER_TRACKING)
18. # define use_awaitable \
19.   boost::asio::use_awaitable_t(__FILE__, __LINE__, __PRETTY_FUNCTION__)
20. #endif
21.
22. awaitable<void> echo(tcp::socket socket)
23. {
24.     try
25.     {
26.         char data[1024];
27.         for (;;)
28.         {
29.             std::size_t n = co_await socket.async_read_some(boost::asio::buffer(data), use_await
    able);
30.             co_await async_write(socket, boost::asio::buffer(data, n), use_awaitable);
31.         }
32.     }
33.     catch (std::exception& e)
34.     {
35.         std::printf("echo Exception: %s\n", e.what());
36.     }
37. }
38.
39. awaitable<void> listener()
40. {
41.     auto executor = co_await this_coro::executor;
```

```
    tcp::acceptor acceptor(executor, { tcp::v4(), 10086 });
    for (;;)
    {
        tcp::socket socket = co_await acceptor.async_accept(use_awaitable);
        co_spawn(executor, echo(std::move(socket)), detached);
    }
}

int main()
{
    try
    {
        boost::asio::io_context io_context(1);

        boost::asio::signal_set signals(io_context, SIGINT, SIGTERM);
        signals.async_wait([&](auto, auto) { io_context.stop(); });

        co_spawn(io_context, listener(), detached);

        io_context.run();
    }
    catch (std::exception& e)
    {
        std::printf("Exception: %s\n", e.what());
    }
}
```

1 我们用awaitable<void>声明了一个函数，那么这个函数就变为可等待的函数了，比如 `listener` 被添加 `awaitable<void>` 之后，就可以被协程调用和等待了。
2 `co_spawn` 表示启动一个协程，参数分别为调度器，执行的函数，以及启动方式, 比如我们启动了一个协程，deatched表示将协程对象分离出来，这种启动方式可以启动多个协程，他们都是独立的，如何调度取决于调度器，在用户的感知上更像是线程调度的模式，类似于并发运行，其实底层都是串行的。

```
co_spawn(io_context, listener(), detached);
```

我们启动了一个协程，执行listener中的逻辑，listener内部co_await 等待 acceptor接收连接，如果没有连接到来则挂起协程。执行之后的 `io_context.run()` 逻辑。所以协程实际上是在一个线程中串行调度的，只是感知上像是并发而已。
3 当acceptor接收到连接后，继续调用co_spawn启动一个协程，用来执行echo逻辑。echo

逻辑里也是通过co_wait的方式接收和发送数据的，如果对端不发数据，执行echo的协程就会挂起，另一个协程启动，继续接收新的连接。当没有连接到来，接收新连接的协程挂起，如果所有协程都挂起，则等待新的就绪事件(对端发数据，或者新连接)到来唤醒。

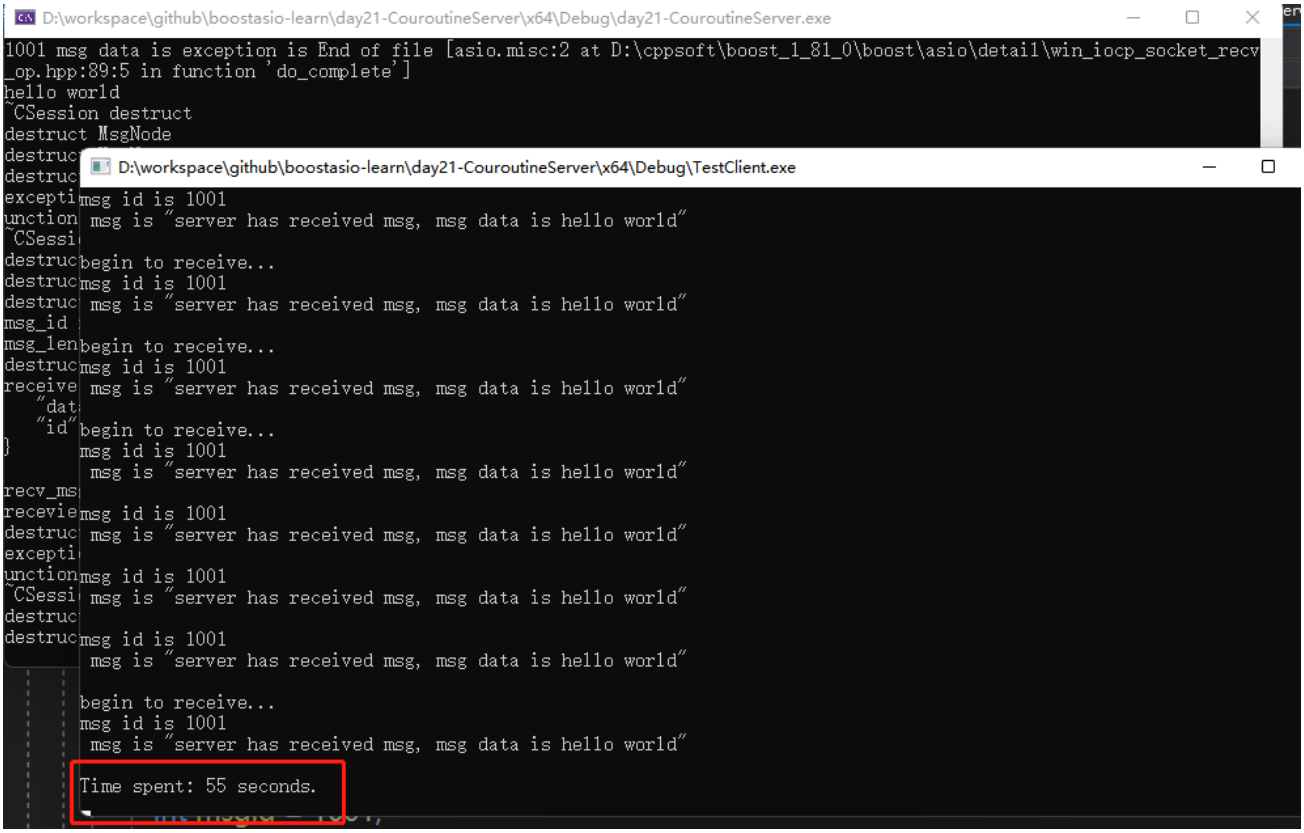
改进服务器

我们可以利用协程改进服务器编码流程，用一个iocontext管理绑定acceptor用来接收新的连接，再用一个iocontext或以IOServicePool的方式管理连接的收发操作，在每个连接的接收数据时改为启动一个协程，通过顺序的方式读取收到的数据

```
1. void CSession::Start() {
2.     auto shared_this = shared_from_this();
3.     //开启接收协程
4.     co_spawn(_io_context, [=]()->awaitable<void> {
5.         try {
6.             for (;!_b_close;) {
7.                 _recv_head_node->Clear();
8.                 std::size_t n = co_await boost::asio::async_read(_socket,
9.                     boost::asio::buffer(_recv_head_node->_data, HEAD_TOTAL_LEN),
10.                    use_awaitable);
11.
12.                if (n == 0) {
13.                    std::cout << "receive peer closed" << endl;
14.                    Close();
15.                    _server->ClearSession(_uuid);
16.                    co_return;
17.                }
18.
19.                //获取头部MSGID数据
20.                short msg_id = 0;
21.                memcpy(&msg_id, _recv_head_node->_data, HEAD_ID_LEN);
22.                //网络字节序转化为本地字节序
23.                msg_id = boost::asio::detail::socket_ops::network_to_host_short(msg_id);
24.                std::cout << "msg_id is " << msg_id << endl;
25.                //id非法
26.                if (msg_id > MAX_LENGTH) {
27.                    std::cout << "invalid msg_id is " << msg_id << endl;
28.                    _server->ClearSession(_uuid);
29.                    co_return;
30.                }
31.                short msg_len = 0;
32.                memcpy(&msg_len, _recv_head_node->_data + HEAD_ID_LEN, HEAD_DATA_LEN);
33.                //网络字节序转化为本地字节序
34.                msg_len = boost::asio::detail::socket_ops::network_to_host_short(msg_len);
35.                std::cout << "msg_len is " << msg_len << endl;
36.                //长度非法
37.                if (msg_len > MAX_LENGTH) {
38.                    std::cout << "invalid data length is " << msg_len << endl;
39.                    _server->ClearSession(_uuid);
40.                    co_return;
41.                }
42.
```

```
43.             _recv_msg_node = make_shared<RecvNode>(msg_len, msg_id);
44.             //读出包体
45.             n = co_await boost::asio::async_read(_socket,
46.                 boost::asio::buffer(_recv_msg_node->_data, _recv_msg_node->_total_len), use_
    awaitable);
47.
48.             if (n == 0) {
49.                 std::cout << "receive peer closed" << endl;
50.                 Close();
51.                 _server->ClearSession(_uuid);
52.                 co_return;
53.             }
54.
55.             _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
56.             cout << "receive data is " << _recv_msg_node->_data << endl;
57.             //投递给逻辑线程
58.             LogicSystem::GetInstance().PostMsgToQue(make_shared<LogicNode>(shared_from
    _this(), _recv_msg_node));
59.         }
60.
61.     }
62.     catch (std::exception& e) {
63.         std::cout << "exception is " << e.what() << endl;
64.         Close();
65.         _server->ClearSession(_uuid);
66.     }
67.     }, detached);
68.
69. }
```

其余的逻辑和之前大体相同，测试了一下在一个iocontext负责接收新连接，一个iocontext负责接收数据和发送数据的情况下，客户端创建100个连接，收发500次，总用时为55s



浏览(3386) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(550) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ^ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)